

LAPORAN PRAKTIKUM 2
PROGRAMMING LANGUAGE PRAGMATICS
PARADIGMA PEMROGRAMAN: IMPERATIVE, OBJECT ORIENTED,
FUNCTIONAL, DAN LOGIC

Laporan Ini Disusun Untuk Memenuhi Tugas Praktikum 2
Mata Kuliah Programming Language Pragmatics



Disusun Oleh :
Ghaisan Khoirul Badruzaman
251524048
2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Lingkungan dan Cara Menjalankan	3
BAGIAN YANG DILENGKAPI	5
PROGRAM 1: PARADIGMA IMPERATIVE	7
3.1 Kode yang Dilengkapi	7
3.2 Keluaran Program	8
3.3 Tabel Pengamatan	9
3.4 Pertanyaan Pengayaan	10
PROGRAM 2: PARADIGMA OBJECT ORIENTED	13
4.1 Kode yang Dilengkapi	13
4.2 Keluaran Program	14
4.3 Tabel Pengamatan	14
4.4 Pertanyaan Pengayaan	16
PROGRAM 3: PARADIGMA FUNCTIONAL DENGAN HASKELL	18
5.1 Kode yang Dilengkapi	18
5.2 Keluaran Program	18
5.3 Tabel Pengamatan	19
5.4 Pertanyaan Pengayaan	21
5.5 Catatan Presisi Float	22
PROGRAM 4: LOGIC PROGRAMMING DENGAN SWI-PROLOG	23
6.1 Query Nomor 13 dan 14	23
6.2 Keluaran Program	23
6.3 Tabel Pengamatan	24
6.4 Pertanyaan Pengayaan	26
LESSON LEARNED PROGRAM 1 SAMPAI PROGRAM 4	28

PENDAHULUAN

1.1 Latar Belakang

Praktikum 2 memakai satu persoalan yang sama untuk empat paradigma sekaligus. Datanya 30 nilai mahasiswa dengan komponen tugas, UTS, dan UAS, lalu nilai akhirnya dihitung dengan bobot 30 persen tugas, 30 persen UTS, dan 40 persen UAS. Di atas data itu dilakukan pencarian, penyaringan, pengurutan, agregasi, dan pencarian nilai tertinggi.

Karena persoalannya dibuat sama, yang dibandingkan bukan hasilnya melainkan cara keempat bahasa menyelesaikannya. Program 1 memakai C++ dengan gaya imperative, Program 2 memakai C++ dengan gaya object oriented, Program 3 memakai Haskell yang functional, dan Program 4 memakai SWI-Prolog yang logic. Konsep binding, scope, lifetime, dan state yang dibahas di praktikum 1 dipakai lagi sebagai alat untuk membandingkan keempatnya.

1.2 Tujuan

Praktikum ini bertujuan untuk:

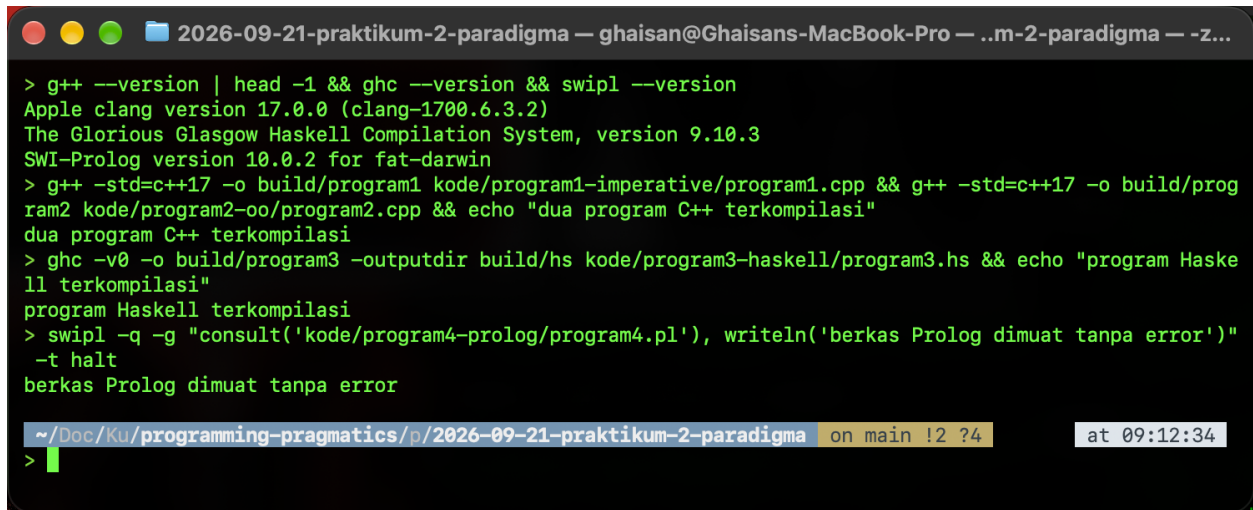
1. Melengkapi bagian program yang sengaja dikosongkan pada keempat berkas, yaitu fungsi pencarian dan penyaringan di Program 1 sampai 3 serta dua query gabungan di Program 4.
2. Menjalankan keempat program dan mencatat keluarannya sebagai bukti.
3. Mengisi tabel pengamatan untuk aspek binding, scope, lifetime, state, dan aspek lain yang diminta tiap program, dengan jawaban yang diambil dari hasil percobaan sendiri.
4. Menjawab pertanyaan pengayaan tiap program dan menutupnya dengan lesson learned yang membandingkan keempat paradigma.

1.3 Lingkungan dan Cara Menjalankan

Seluruh program dijalankan di macOS dengan toolchain berikut: g++ dari Apple clang 17.0.0 dengan standar C++17, GHC 9.10.3 lewat ghcup, dan SWI-Prolog 10.0.2. Program 1 dan 2 dikompilasi jadi berkas biner di folder build, Program 3 bisa dijalankan langsung dengan runghc atau dikompilasi dengan ghc, dan Program 4 dimuat dengan swipl. Perintah lengkapnya ada di berkas run.txt.

Program 1 dan 2 menunya interaktif, jadi supaya keluarannya bisa direkam dan diulang, inputnya disiapkan di berkas input-demo.txt lalu dialihkan ke program. Urutan yang dipakai sama untuk keduanya, yaitu mencari NIM 24013, mencari nama Maya, menyaring nilai minimal 80, lalu keluar.

Kompilasi keempat program dan pemuatan berkas Prolog:



```
2026-09-21-praktikum-2-paradigma — ghaisan@Ghaisans-MacBook-Pro — ..m-2-paradigma — -z...  
> g++ --version | head -1 && ghc --version && swipl --version  
Apple clang version 17.0.0 (clang-1700.6.3.2)  
The Glorious Glasgow Haskell Compilation System, version 9.10.3  
SWI-Prolog version 10.0.2 for fat-darwin  
> g++ -std=c++17 -o build/program1 kode/program1-imperative/program1.cpp && g++ -std=c++17 -o build/program2 kode/program2-oo/program2.cpp && echo "dua program C++ terkompilasi"  
dua program C++ terkompilasi  
> ghc -v0 -o build/program3 -outputdir build/hs kode/program3-haskell/program3.hs && echo "program Haskell terkompilasi"  
program Haskell terkompilasi  
> swipl -q -g "consult('kode/program4-prolog/program4.pl'), writeln('berkas Prolog dimuat tanpa error')"  
-t halt  
berkas Prolog dimuat tanpa error  
~/Doc/Ku/programming-pragmatics/p/2026-09-21-praktikum-2-paradigma on main !2 ?4 at 09:12:34  
>
```

Gambar 1.1: Versi toolchain serta kompilasi keempat program

BAGIAN YANG DILENGKAPI

Kode dari dokumen disalin apa adanya. Yang diubah hanya bagian yang memang dikosongkan dosen, ditambah satu baris di Program 1 yang membuat kodenya tidak bisa dikompilasi. Pesan error kode asli sebelum dilengkapi direkam di berkas hasil/kompilasi-asli.txt.

Tabel 2.1: Bagian yang dilengkapi pada keempat program

Program	Bagian yang dilengkapi	Alasan
1 Imperative	<code>searchNIM(Mahasiswa[], int, string)</code>	Komentar // TAMBAHKAN FUNGSI SEARCH BERDASARKAN NIM memang kosong. Dibat linear search karena array sudah diurutkan berdasarkan nilai, bukan berdasarkan NIM, jadi binary search tidak bisa dipakai.
1 Imperative	<code>searchNama(Mahasiswa[], int, string)</code>	Sama, bagiannya kosong. Cocokkan string nama persis.
1 Imperative	<code>searchNilai(Mahasiswa[], int, float)</code>	Sama, bagiannya kosong. Dibat menampilkan semua mahasiswa dengan nilai lebih besar atau sama dengan batas, mengikuti keterangan di Program 2 yang menulis "Menampilkan nilai >= batas".
1 Imperative	<code>case 1: searchNIM(NIM); di menuFilter()</code>	Ini perbaikan wajib. NIM tidak pernah dideklarasikan di mana pun, jadi kode dosen gagal dikompilasi dengan pesan error: use of undeclared identifier 'NIM'. Diganti jadi blok case yang membaca input dulu lalu memanggil <code>searchNIM(mahasiswa, jumlah, nim)</code> , meniru gaya <code>menuFilter()</code> di Program 2 supaya konsisten.
1 Imperative	<code>case 2 dan case 3 di menuFilter()</code>	Aslinya hanya komentar //fungsi search by Nama dan //fungsi search by Nilai, menunya jadi tidak melakukan apa-apa.
2 Object oriented	<code>filterNIM(string)</code>	Sudah dipanggil di <code>menuFilter()</code> tapi methodnya belum ada, jadi gagal kompilasi dengan error: use of undeclared identifier 'filterNIM'.
2 Object oriented	<code>filterNama(string)</code>	Sama, dipanggil tapi belum ada.
2 Object oriented	<code>filterNilai(float)</code>	Sama, dipanggil tapi belum ada. Memakai <code>getNilai()</code> supaya tetap lewat getter, tidak menyentuh atribut private langsung.
3 Haskell	<code>searchNIM :: String -> [Mahasiswa] -> [Mahasiswa]</code>	Dipanggil di main tapi belum didefinisikan, GHC bilang Variable not in scope: searchNIM. Dibat dengan filter supaya tetap murni dan mengembalikan list, sesuai cara main memakainya lewat <code>mapM_ tampilkan</code> .
3 Haskell	<code>searchNama :: String -> [Mahasiswa] -> [Mahasiswa]</code>	Sama, belum didefinisikan.
3 Haskell	<code>filterNilai :: Float -> [Mahasiswa] -> [Mahasiswa]</code>	Sama, belum didefinisikan. Urutan parameter mengikuti pemanggilan <code>filterNilai 80 mahasiswa</code> .
4 Prolog	Query nomor 13	Dokumen menulis "Buatkan sintaksnya". Jawabannya <code>?- lulus(NIM), \+ grade(NIM, a) .</code> Negasi pakai <code>\+</code> karena Prolog tidak punya "tidak mendapat A" sebagai fakta, jadi harus pakai negation as failure.
4 Prolog	Query nomor 14	Jawabannya <code>?- perlu_bimbingan(NIM), nilai_akhir(NIM, Nilai), Nilai < 60. nilai_akhir</code> dipanggil lagi supaya nilainya ikut ter-binding dan bisa dibandingkan dengan 60.
2 Object oriented	Komentar // Nilai tertinggi -> terendah	Di dokumen komentar ini memakai karakter panah unicode. Diganti jadi <code>-></code> biasa supaya seluruh berkas tetap ASCII dan tidak bermasalah waktu dipindah antar editor. Ini cuma komentar, tidak mengubah jalannya program.

Program	Bagian yang dilengkapi	Alasan
4 Prolog	Predikat <code>demo</code> dan <code>demo_pengamatan</code> (BAGIAN 6 dan 7)	Tambahan sendiri, bukan bagian yang dikosongkan dosen. Dipakai supaya keluaran ke-14 query bisa direkam ke berkas, karena toplevel SWI-Prolog susah dipakai kalau inputnya dialihkan dari berkas. Semua fakta dan rule dosen tidak disentuh.

Selain itu ada tujuh berkas kecil di kode/eksperimen yang dibuat sendiri untuk menjawab baris tabel pengamatan yang meminta mencoba sesuatu, misalnya memakai variabel `i` setelah blok `for`. Tiga di antaranya memang sengaja dibuat gagal dikompilasi karena hasil kompilasinya yang ditanyakan.

Tabel 2.2: Status kompilasi dan eksekusi

Program	Kompilasi	Eksekusi	Keluaran
1 Imperative	Berhasil, tanpa warning	Berhasil, input dari <code>kode/program1-imperative/input-demo.txt</code>	<code>hasil/program1.txt</code>
2 Object oriented	Berhasil, tanpa warning	Berhasil, input dari <code>kode/program2-oo/input-demo.txt</code>	<code>hasil/program2.txt</code>
3 Haskell	Berhasil, <code>runghc</code> maupun <code>ghc</code> sama hasilnya	Berhasil	<code>hasil/program3.txt</code>
4 Prolog	Berhasil dimuat tanpa warning	Berhasil, <code>demo</code> dan <code>demo_pengamatan</code>	<code>hasil/program4.txt</code> , <code>hasil/program4-pengamatan.txt</code>

Hasil utamanya sama di keempat program. Nilai tertinggi Maya dengan NIM 24013 sebesar 94.50, terendah Putra dengan NIM 24016 sebesar 57.40, rata rata kelas 77.80667, dan ada 10 mahasiswa yang memperoleh grade A.

PROGRAM 1: PARADIGMA IMPERATIVE

3.1 Kode yang Dilengkapi

Tiga fungsi pencarian di Program 1 dikosongkan dosen. Ketiganya ditulis dengan linear search karena array sudah diurutkan berdasarkan nilai akhir, bukan berdasarkan NIM atau nama, sehingga binary search tidak bisa dipakai. Pola ketiganya sama: satu penghitung ketemu, satu loop, lalu pesan kalau tidak ada yang cocok.

Kode 3.1. Pencarian berdasarkan NIM (program1.cpp)

```
void searchNIM(Mahasiswa mahasiswa[], int jumlah, string kunci) {

    int ketemu = 0;

    for (int i = 0; i < jumlah; i++) {

        if (mahasiswa[i].nim == kunci) {

            tampilkanMahasiswa(mahasiswa[i]);

            ketemu++;

        }

    }

    if (ketemu == 0) {

        cout << "Data dengan NIM " << kunci
             << " tidak ditemukan.\n";

    }

}
```

Kode 3.2. Pencarian berdasarkan nilai minimum (program1.cpp)

```
void searchNilai(Mahasiswa mahasiswa[], int jumlah, float batas) {

    int ketemu = 0;

    for (int i = 0; i < jumlah; i++) {

        if (mahasiswa[i].nilaiAkhir >= batas) {

            tampilkanMahasiswa(mahasiswa[i]);

            ketemu++;

        }

    }

    if (ketemu == 0) {

        cout << "Tidak ada mahasiswa dengan nilai >= "
             << batas << " .\n";

    } else {

        cout << "Jumlah data: " << ketemu << endl;

    }

}
```

```
}
```

```
// =====
```

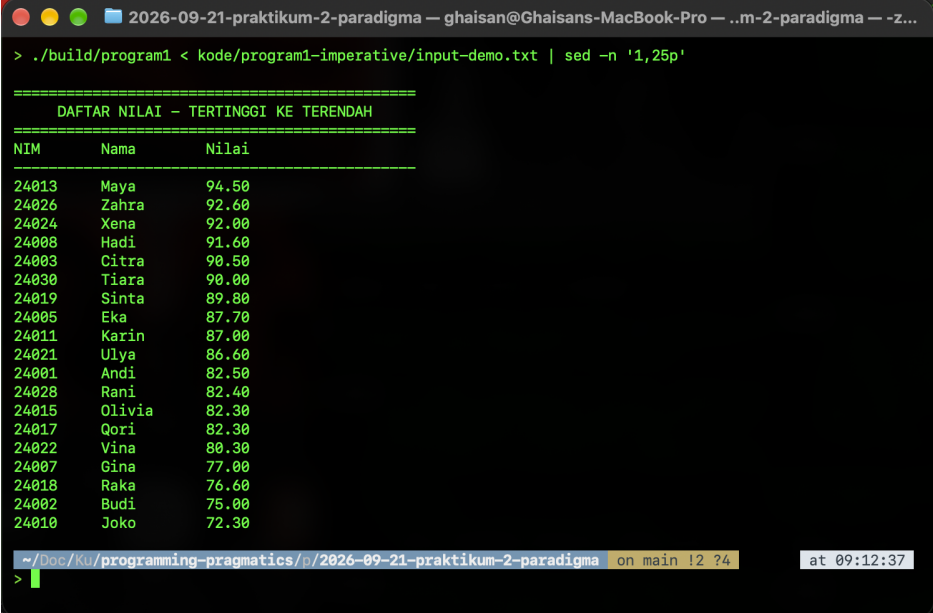
Di menu, baris case 1 pada kode dosen tertulis searchNIM(NIM) padahal NIM tidak pernah dideklarasikan, sehingga g++ menolak dengan pesan use of undeclared identifier NIM. Ketiga cabang menu ditulis ulang supaya membaca input dulu baru memanggil fungsinya, mengikuti gaya menu Program 2.

Kode 3.3. Cabang menu yang diperbaiki (program1.cpp)

```
switch (pilihan) {  
  
    case 1: {  
  
        string nim;  
  
        cout << "Masukkan NIM: ";  
        cin >> nim;  
  
        cout << "\nHasil pencarian:\n";  
  
        searchNIM(mahasiswa, jumlah, nim);  
  
        break;  
    }  
}
```

3.2 Keluaran Program

Program dijalankan dengan input dari berkas. Bagian pertama keluarannya adalah daftar nilai yang sudah diurutkan dari tertinggi ke terendah:



```
2026-09-21-praktikum-2-paradigma - ghaisan@Ghaisans-MacBook-Pro - ..m-2-paradigma - -z...  
> ./build/program1 < kode/program1-imperative/input-demo.txt | sed -n '1,25p'  
  
=====  
DAFTAR NILAI - TERTINGGI KE TERENDAH  
=====  
NIM      Nama      Nilai  
-----  
24013    Maya      94.50  
24026    Zahra     92.60  
24024    Xena      92.00  
24008    Hadi      91.60  
24003    Citra     90.50  
24030    Tiara     90.00  
24019    Sinta     89.80  
24005    Eka       87.70  
24011    Karin     87.00  
24021    Ulya      86.60  
24001    Andi      82.50  
24028    Rani      82.40  
24015    Olivia    82.30  
24017    Qori      82.30  
24022    Vina      80.30  
24007    Gina      77.00  
24018    Raka      76.60  
24002    Budi      75.00  
24010    Joko      72.30  
  
~/Doc/Ku/programming-pragmatics/p/2026-09-21-praktikum-2-paradigma on main !2 ?4 at 09:12:37  
>
```

Gambar 3.1: Daftar nilai Program 1 setelah bubble sort

Bagian berikutnya adalah menu filter yang memakai ketiga fungsi pencarian tadi:

```

> ./build/program1 < kode/program1-imperative/input-demo.txt | sed -n '37,60p'

=====
MENU FILTER
=====
1. Search berdasarkan NIM
2. Search berdasarkan Nama
3. Search berdasarkan Nilai
4. Kembali
Pilihan: Masukkan NIM:
Hasil pencarian:
24013    Maya    94.50

Filter lagi? (1 = Ya, 0 = Tidak):
=====
MENU FILTER
=====
1. Search berdasarkan NIM
2. Search berdasarkan Nama
3. Search berdasarkan Nilai
4. Kembali
Pilihan: Masukkan Nama:
Hasil pencarian:
24013    Maya    94.50

~/Doc/Ku/programming-pragmatics/p/2026-09-21-praktikum-2-paradigma on main !2 ?4 at 09:12:50
>

```

Gambar 3.2: Hasil pencarian berdasarkan NIM dan nama

3.3 Tabel Pengamatan

Tabel 3.1: Hasil pengamatan Program 1

Aspek	Pengamatan spesifik	Jawaban
Binding	Identifikasi kapan <code>i</code> pertama kali memperoleh nilai. Identifikasi kapan <code>mahasiswa[i].nilaiAkhir</code> memperoleh nilai.	<code>i</code> memperoleh nilai saat bagian inisialisasi <code>int i = 0</code> dijalankan, yaitu sekali setiap kali loop dimasuki. Kalau fungsinya dipanggil lagi, <code>i</code> dibuat dan diberi nilai lagi dari awal. <code>mahasiswa[i].nilaiAkhir</code> baru memperoleh nilai hasil hitungan di dalam <code>hitungNilai()</code> . Sebelum itu isinya 0.00, terlihat di <code>hasil/eksperimen.txt</code> bagian [1]: Andi 0.00 dan Maya 0.00, lalu di bagian [2] berubah jadi 82.50 dan 94.50. Nilai 0.00 itu datang dari aggregate initialization yang cuma diisi lima anggota, anggota keenam otomatis dinolkan.
Binding	Perhatikan apakah nilai <code>nilaiAkhir</code> dapat diubah lagi setelah assignment pertama.	Bisa. Bindingnya mutable, yang terikat ke nama itu lokasi memori, bukan nilai tetap. Buktinya di <code>hasil/eksperimen.txt</code> bagian [6]: <code>sortNilai()</code> dipanggil dulu waktu semua nilai masih 0.00, lalu <code>hitungNilai()</code> menempa isinya jadi 57.40, 94.50, dan 82.50. <code>sortNilai()</code> sendiri juga menempa <code>nilaiAkhir</code> setiap kali menukar dua elemen.
Scope	Coba gunakan <code>i</code> setelah blok <code>for</code> . Apa yang terjadi?	Gagal dikompilasi. Dicoba di <code>kode/eksperimen/p1-scope.cpp</code> , g++ menjawab error: use of undeclared identifier ' <code>i</code> ' di baris 18. <code>i</code> dideklarasikan di bagian inisialisasi <code>for</code> , jadi scopenya cuma header dan body loop itu. Begitu keluar kurung kurawal, namanya tidak dikenal lagi.

Aspek	Pengamatan spesifik	Jawaban
Scope	Coba gunakan variabel lokal di dalam fungsi <code>hitungNilai()</code> dari <code>main()</code> . Apa yang terjadi?	Gagal dikompilasi juga. Di berkas yang sama, variabel <code>penghitung</code> milik <code>hitungNilai()</code> dipakai dari <code>main()</code> dan g++ menjawab <code>error: use of undeclared identifier 'penghitung'</code> di baris 19. Scope variabel lokal berhenti di kurung kurawal fungsinya, fungsi lain tidak bisa melihat namanya.
Lifetime	Tambahkan <code>int temp;</code> di dalam blok <code>if</code> . Amati kapan variabel tersebut dapat digunakan.	<code>temp</code> cuma hidup dari baris deklarasinya sampai blok <code>if</code> ditutup. Di <code>kode/eksperimen/pl-lifetime.cpp</code> , di dalam blok nilainya tercetak normal, begitu dipakai di luar blok g++ menjawab <code>error: use of undeclared identifier 'temp'</code> . Di <code>sortNilai()</code> hal yang sama terjadi: Mahasiswa <code>temp</code> dibuat ulang setiap kali kondisi <code>if</code> benar, lalu dibuang lagi begitu blok selesai.
Lifetime	Bandingkan variabel lokal fungsi dengan array mahasiswa. Mana yang hidup lebih lama?	Array mahasiswa yang hidup lebih lama. Dia dideklarasikan di <code>main()</code> jadi umurnya selama <code>main()</code> berjalan, sedangkan <code>i</code> , <code>j</code> , dan <code>temp</code> di <code>sortNilai()</code> lahir dan mati berkali-kali selama fungsi itu jalan. Buktinya array yang sama masih bisa dipakai <code>menuFilter()</code> sampai program mau selesai, sedangkan <code>temp</code> sudah tidak ada namanya begitu blok <code>if</code> lewat.
State	Catat nilai <code>nilaiAkhir</code> sebelum dan sesudah <code>hitungNilai()</code> .	Dari <code>hasil/eksperimen.txt</code> : sebelum, mahasiswa[0] Andi 0.00 dan mahasiswa[12] Maya 0.00. Sesudah, Andi 82.50 dan Maya 94.50. Isinya berubah padahal variabelnya itu-itu juga, jadi yang berubah state objeknya, bukan objek baru.
State	Catat posisi dua mahasiswa sebelum dan sesudah <code>sortNilai()</code> .	Sebelum sort, Andi 24001 ada di indeks 0 dan Maya 24013 di indeks 12. Sesudah sort, Andi pindah ke indeks 10 dan Maya ke indeks 0. Array yang sama, isinya yang digeser. <code>sortNilai()</code> juga tidak mengembalikan apa-apa, jadi satu-satunya efek yang terlihat memang perubahan state array itu.
Execution	Tentukan urutan eksekusi: inialisasi, kondisi, body, increment.	Urutannya inialisasi sekali di awal, lalu kondisi, kalau benar body, lalu increment, lalu balik ke kondisi lagi. Loop berhenti waktu kondisi salah dan body untuk iterasi itu tidak dijalankan. Di <code>hasil/eksperimen.txt</code> bagian [5] untuk <code>for (int i = 0; i < 3; i++)</code> body jalan dengan <code>i = 0, 1, dan 2</code> , lalu pada <code>i = 3</code> kondisi <code>3 < 3</code> salah sehingga body tidak dicetak.
Execution	Identifikasi statement yang menyebabkan perubahan state.	Ada dua tempat. Pertama <code>assignment mahasiswa[i].nilaiAkhir = ...</code> di <code>hitungNilai()</code> . Kedua tiga assignment tukar posisi di <code>sortNilai()</code> , yaitu <code>temp = mahasiswa[j]</code> , <code>mahasiswa[j] = mahasiswa[j+1]</code> , dan <code>mahasiswa[j+1] = temp</code> . Selain itu <code>i++</code> , <code>j++</code> , dan <code>cin >></code> pilihan juga mengubah state, tapi cuma state variabel lokal dan bukan data mahasiswanya. Statement <code>cout</code> tidak mengubah state data sama sekali.
Abstraction	Identifikasi fungsi yang digunakan untuk menyembunyikan detail operasi, misalnya <code>hitungNilai()</code> dan <code>sortNilai()</code> .	<code>hitungNilai()</code> menyembunyikan bobot 30, 30, 40 sehingga <code>main()</code> cukup tahu "hitung nilainya" tanpa tahu rumusnya. <code>sortNilai()</code> menyembunyikan bubble sort dengan dua loop bersarang dan variabel <code>temp</code> . <code>tampilkanMahasiswa()</code> menyembunyikan <code>setw</code> dan <code>setprecision</code> . Fungsi <code>search</code> yang saya tambah juga begitu, <code>menuFilter()</code> tidak perlu tahu pencariannya linear. Yang bocor dari abstraksi ini: ketiganya mengubah array yang dioper, jadi pemanggil tetap harus tahu bahwa datanya berubah.

3.4 Pertanyaan Pengayaan

1. Jika nilai `nilaiAkhir` dihitung ulang setelah nilai UAS berubah, bagian program mana yang harus dieksekusi kembali? Mengapa?

`hitungNilai()` harus dipanggil lagi, lalu `sortNilai()` juga, baru tampilannya dicetak ulang. Alasannya `nilaiAkhir` disimpan sebagai data, bukan dihitung tiap kali dibutuhkan. Jadi kalau `uas` berubah tapi `hitungNilai()` tidak dipanggil, `nilaiAkhir` masih memegang angka lama

dan seluruh program ikut salah. `sortNilai()` juga harus diulang karena urutannya dibuat berdasarkan nilai lama. Ini yang biasa disebut data turunan tidak sinkron dengan data sumbernya.

2. Apa yang terjadi jika variabel `nilaiAkhir` dibuat sebagai variabel lokal di dalam fungsi `hitungNilai()` dibandingkan sebagai atribut dari setiap mahasiswa?

Kalau jadi variabel lokal, dia mati begitu `hitungNilai()` selesai, jadi `sortNilai()`, `tampilkanMahasiswa()`, dan `searchNilai()` tidak punya nilai untuk dibandingkan. Efeknya nilainya harus dihitung ulang di setiap fungsi yang butuh, dan rumus 30/30/40 tersebar di banyak tempat. Sebagai atribut, nilainya hidup selama array hidup dan cukup dihitung sekali, tapi risikonya jadi bisa basi seperti jawaban nomor 1.

3. Mengapa perubahan urutan dua statement berikut dapat menghasilkan state yang berbeda? `hitungNilai(); sortNilai();` dan `sortNilai(); hitungNilai();`

Karena `sortNilai()` membaca `nilaiAkhir` untuk menentukan urutan, sedangkan `hitungNilai()` yang mengisinya. Kalau sort duluan, yang dibandingkan masih 0.00 semua sehingga tidak ada penukaran yang berarti, baru setelah itu nilainya diisi ke posisi yang sudah salah. Saya coba di `kode/eksperimen/p1-state.cpp` bagian [6] dengan tiga data: hasilnya indeks 0 Putra 57.40, indeks 1 Maya 94.50, indeks 2 Andi 82.50, jelas tidak terurut. Bandingkan dengan urutan yang benar di `hasil/program1.txt` yang diawali Maya 94.50. Jadi di paradigma imperative urutan statement ikut menentukan hasil, karena tiap statement bekerja di atas state yang ditinggalkan statement sebelumnya.

4. Pada Bubble Sort, mengapa diperlukan variabel sementara `temp`? Apa hubungan penggunaan `temp` dengan konsep mutable state?

Karena penukaran dilakukan dengan menimpa isi kotak. Begitu `mahasiswa[j] = mahasiswa[j+1]` dijalankan, isi lama `mahasiswa[j]` hilang, jadi harus disalin dulu ke `temp`. Kalau tidak, dua elemen jadi kembar. Ini murni akibat mutable state: yang dipindahkan adalah isi lokasi memori, bukan membuat list baru. Di Haskell masalah ini tidak ada karena `sortBy` menghasilkan list baru, tidak menimpa apa-apa.

5. Jika `i` pada `for` memiliki scope lokal, bagaimana programmer dapat mengakses hasil iterasi setelah loop selesai?

Datanya harus disimpan di variabel yang scopenya lebih luas, dideklarasikan sebelum loop. Cara lain, `i` dideklarasikan di luar `for` lalu loopnya ditulis `for (i = 0; i < n; i++)`, tapi ini bikin `i` hidup lebih lama dari yang perlu dan gampang kepakai tidak sengaja. Pola yang saya pakai di `searchNIM()` dan `searchNilai()` adalah membuat variabel ketemu sebelum loop, lalu menambahnya di dalam loop. Hasilnya terlihat di `hasil/program1.txt` sebagai baris Jumlah data: 15 yang dicetak setelah loop selesai.

6. Apa risiko jika sebuah fungsi mengubah array mahasiswa secara langsung dibandingkan jika fungsi hanya mengembalikan hasil baru?

Risikonya data aslinya hilang dan semua bagian program lain ikut terpengaruh tanpa diberi tahu. Setelah `sortNilai()`, urutan asli sesuai NIM tidak bisa dikembalikan lagi karena tidak ada salinannya. Fungsi yang mengembalikan hasil baru lebih aman, data awal tetap utuh dan bisa dipakai untuk keperluan lain, tapi butuh memori lebih banyak. Program 3 memakai cara ini, di `hasil/eksperimen.txt` bagian [3] dan [6] terlihat list awalnya tetap empat elemen dengan urutan yang sama walaupun sudah difilter dan disort.

7. Bagaimana perubahan state memengaruhi debugging? Misalnya, jika nilai mahasiswa berubah secara tidak terduga, informasi apa yang perlu dilacak?

Debugging jadi susah karena melihat kode saja tidak cukup, harus tahu juga statement mana yang sudah jalan sebelumnya. Yang perlu dilacak: fungsi apa saja yang menerima array itu, urutan pemanggilannya, indeks yang disentuh, dan nilai variabel sebelum serta sesudah tiap fungsi. Cara paling sederhana yang saya pakai ya seperti di eksperimen, cetak nilainya sebelum dan sesudah tiap fungsi. Contohnya indeks Andi berubah dari 0 jadi 10 setelah `sortNilai()`, kalau tidak sadar itu terjadi dan kita masih memakai indeks 0 untuk mengambil Andi, datanya sudah salah tapi program tetap jalan tanpa error.

8. Jika program semakin besar dan banyak fungsi dapat mengubah data mahasiswa, apa konsekuensinya terhadap pemeliharaan program?

Setiap kali ada bug, semua fungsi yang bisa menyentuh data itu jadi tersangka, jadi waktu mencarinya naik terus seiring jumlah fungsi. Menambah fungsi baru juga berisiko karena bisa merusak asumsi fungsi lama, misalnya fungsi lama menganggap array sudah terurut. Urutan pemanggilan di `main()` jadi bagian dari aturan program yang tidak tertulis di mana pun. Program 2 mengurangi masalah ini dengan `private` dan `getter`, Program 3 mengurangi dengan tidak mengubah data sama sekali.

PROGRAM 2: PARADIGMA OBJECT ORIENTED

4.1 Kode yang Dilengkapi

Di Program 2 ketiga filter sudah dipanggil di menu tapi methodnya belum ada, jadi kodenya gagal dikompilasi dengan tiga pesan use of undeclared identifier. Isinya mirip Program 1, bedanya data diakses lewat getter karena atributnya private.

Kode 4.1. Filter berdasarkan NIM di kelas SistemAkademik (program2.cpp)

```
void filterNIM(string kunci) {  
  
    int ketemu = 0;  
  
    for (int i = 0; i < jumlah; i++) {  
  
        if (mahasiswa[i]->getNIM() == kunci) {  
  
            mahasiswa[i]->tampilkan();  
  
            ketemu++;  
        }  
    }  
  
    if (ketemu == 0) {  
  
        cout << "Data dengan NIM " << kunci  
            << " tidak ditemukan.\n";  
    }  
}
```

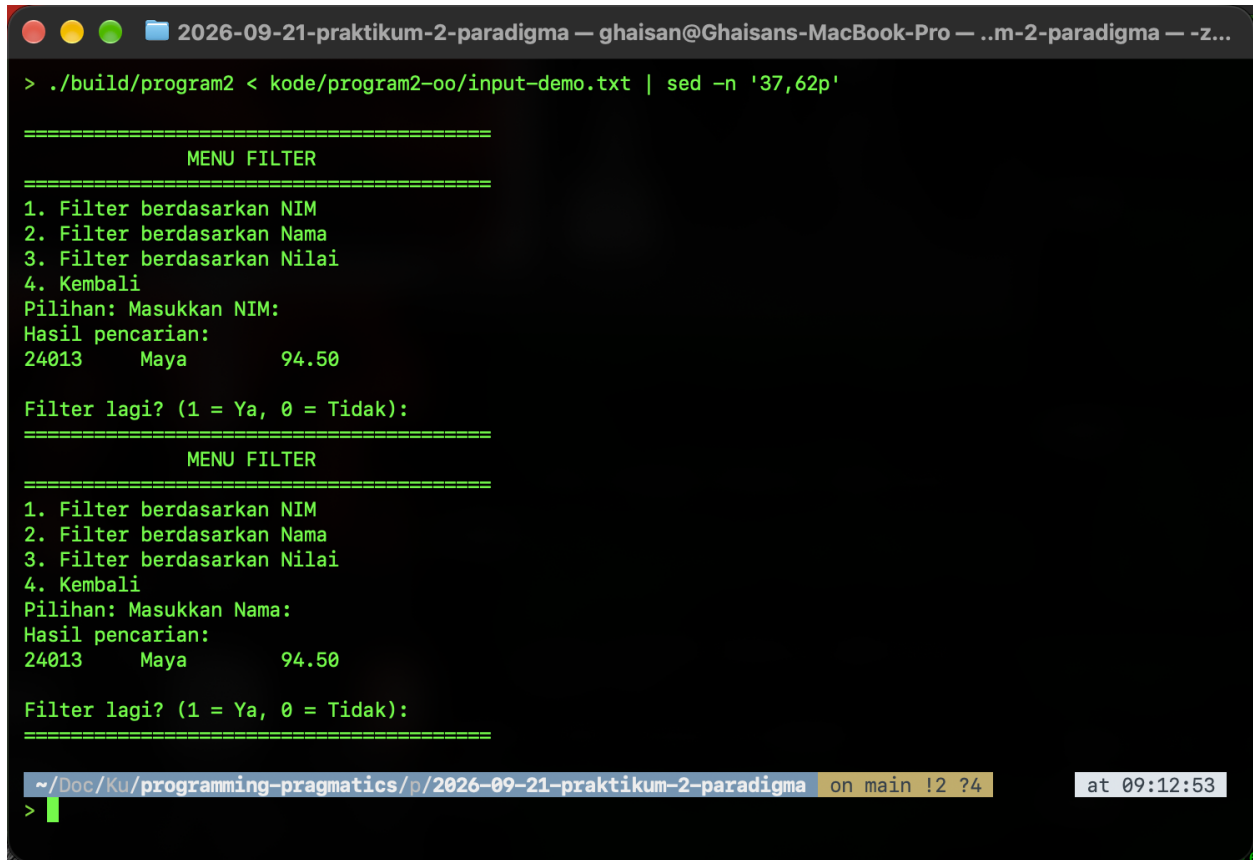
Kode 4.2. Filter berdasarkan nilai minimum (program2.cpp)

```
void filterNilai(float batas) {  
  
    int ketemu = 0;  
  
    for (int i = 0; i < jumlah; i++) {  
  
        if (mahasiswa[i]->getNilai() >= batas) {  
  
            mahasiswa[i]->tampilkan();  
  
            ketemu++;  
        }  
    }  
  
    if (ketemu == 0) {  
  
        cout << "Tidak ada mahasiswa dengan nilai >= "  
            << batas << ".\n";  
    } else {  
  
        cout << "Jumlah data: " << ketemu << endl;  
    }  
}
```

```
// =====
```

4.2 Keluaran Program

Menu filter Program 2 dijalankan dengan urutan input yang sama seperti Program 1:



```
> ./build/program2 < kode/program2-oo/input-demo.txt | sed -n '37,62p'

=====
MENU FILTER
=====
1. Filter berdasarkan NIM
2. Filter berdasarkan Nama
3. Filter berdasarkan Nilai
4. Kembali
Pilihan: Masukkan NIM:
Hasil pencarian:
24013    Maya    94.50

Filter lagi? (1 = Ya, 0 = Tidak):

=====
MENU FILTER
=====
1. Filter berdasarkan NIM
2. Filter berdasarkan Nama
3. Filter berdasarkan Nilai
4. Kembali
Pilihan: Masukkan Nama:
Hasil pencarian:
24013    Maya    94.50

Filter lagi? (1 = Ya, 0 = Tidak):

=====

~/Doc/Ku/programming-pragmatics/p/2026-09-21-praktikum-2-paradigma on main !2 ?4 at 09:12:53
>
```

Gambar 4.1: Hasil filter Program 2 lewat getter

4.3 Tabel Pengamatan

Tabel 4.1: Hasil pengamatan Program 2

Aspek	Pengamatan spesifik	Jawaban
Binding	Kapan objek <code>m</code> dikaitkan dengan instance Mahasiswa?	Saat <code>new Mahasiswa(...)</code> dijalankan. Operator <code>new</code> mengalokasikan memori, constructor mengisinya, lalu alamatnya dikembalikan dan diikat ke pointer <code>m</code> . Di <code>main()</code> hal ini terjadi 30 kali, satu untuk tiap baris <code>sistem.tambahMahasiswa(new Mahasiswa(...))</code> . Bindingnya terjadi saat runtime, bukan saat kompilasi, karena alamatnya baru diketahui waktu program jalan. Di <code>hasil/eksperimen.txt</code> bagian [1] terlihat alamatnya memang beda beda: objek 0 di <code>0x104a118f0</code> , objek 12 di <code>0x104a130c0</code> , objek 29 di <code>0x104a13500</code> .
Binding	Kapan atribut <code>nim</code> , <code>nama</code> , dan <code>nilaiAkhir</code> memperoleh nilai?	<code>nim</code> dan <code>nama</code> memperoleh nilai di baris <code>this->nim = nim;</code> dan <code>this->nama = nama;</code> di dalam constructor. <code>nilaiAkhir</code> sedikit belakangan, yaitu waktu constructor memanggil <code>hitungNilai()</code> di baris terakhirnya. Jadi ketiganya sudah terisi sebelum objeknya dipakai siapa pun, beda dengan Program 1 yang <code>nilaiAkhir</code> -nya sempat 0.00.

Aspek	Pengamatan spesifik	Jawaban
Binding	Identifikasi method yang dipanggil melalui objek m.	Yang dipanggil lewat objek di program ini: <code>hitungNilai()</code> dari dalam constructor, lalu <code>getNIM()</code> , <code>getNama()</code> , dan <code>getNilai()</code> dari <code>filterNIM()</code> , <code>filterNama()</code> , <code>filterNilai()</code> , dan <code>sortNilai()</code> , serta <code>tampilkan()</code> dari <code>tampilkanSemua()</code> dan ketiga filter. Semuanya diakses lewat pointer pakai <code>-></code> , misalnya <code>mahasiswa[i]->getNilai()</code> .
Scope	Apakah <code>nim</code> dapat diakses langsung dari <code>main()</code> ? Mengapa?	Tidak bisa. Dicoba di kode/eksperimen/p2-scope.cpp dan g++ menjawab error: 'nim' is a private member of 'Mahasiswa' ditambah catatan <code>declared private here</code> . Karena <code>nim</code> ada di bagian <code>private</code> , namanya cuma kelihatan dari dalam kelas itu sendiri. Untuk mengambilnya dari luar harus lewat <code>getNIM()</code> .
Scope	Apakah <code>getNilai()</code> dapat dipanggil dari <code>main()</code> ?	Bisa, karena <code>getNilai()</code> ada di bagian <code>public</code> . Di berkas eksperimen yang sama, baris <code>cout << m.getNilai()</code> tidak dipermasalahkan g++, yang error cuma baris <code>m.nim</code> di bawahnya. Jadi satu kelas bisa punya anggota yang scopenya beda beda tergantung labelnya.
Lifetime	Kapan objek <code>m</code> dibuat?	Waktu <code>new Mahasiswa(...)</code> dijalankan, satu per satu sesuai urutan baris di <code>main()</code> . Objeknya ada di heap, jadi tetap hidup walaupun ekspresi yang membuatnya sudah selesai. Di <code>hasil/eksperimen.txt</code> bagian [3] sampai [5] terlihat baris constructor objek 99001, constructor objek 99002, dan constructor objek 99003 muncul tepat saat objeknya dibuat.
Lifetime	Kapan objek <code>m</code> dihancurkan?	Di program dosen objeknya tidak pernah dihancurkan sampai program berakhir, karena tidak ada <code>delete</code> sama sekali. Ini kebocoran memori, cuma tidak kelihatan karena programnya pendek. Di eksperimen saya tambahkan destructor yang mencetak pesan. Hasilnya: objek stack 99001 mencetak destructor objek 99001 tepat saat keluar blok, objek 99002 yang dibuat dengan <code>new</code> tanpa <code>delete</code> tidak pernah mencetak apa apa, dan objek 99003 baru mencetak destructornya setelah <code>delete</code> dipanggil. Ke-30 objek di program asli nasibnya sama dengan 99002.
State	Atribut apa yang berubah ketika <code>hitungNilai()</code> dipanggil?	Cuma <code>nilaiAkhir</code> . Method itu membaca tugas, uts, dan uas tapi tidak menulis ke ketiganya. Dibuktikan di <code>hasil/eksperimen.txt</code> bagian [2]: setelah <code>setUAS(50)</code> dan <code>hitungNilai()</code> dipanggil pada objek indeks 12, nilainya turun dari 94.50 jadi 76.10 sedangkan <code>nim</code> dan namanya tetap.
State	Apakah dua objek <code>Mahasiswa</code> mempunyai state yang sama?	Tidak, masing masing punya state sendiri. Buktinya di eksperimen yang sama: setelah UAS objek indeks 12 diubah jadi 50, objek 0 tetap 82.50 dan objek 29 tetap 90.00, cuma objek 12 yang berubah jadi 76.10. Alamat ketiganya juga berbeda. Yang dipakai bersama cuma kode methodnya, datanya terpisah tiap objek.
Encapsulation	Apa yang terjadi jika <code>private</code> diubah menjadi <code>public</code> ?	Programnya tetap jalan dan hasilnya sama, tapi <code>main()</code> jadi bisa menulis <code>m->nilaiAkhir = 100;</code> tanpa lewat <code>hitungNilai()</code> . Akibatnya <code>nilaiAkhir</code> bisa tidak cocok lagi dengan tugas, uts, dan uas, dan aturan bobot 30/30/40 tidak lagi dijamin kelas. Yang hilang bukan kemampuan, tapi jaminan. Dengan <code>private</code> , satu satunya jalan mengisi <code>nilaiAkhir</code> adalah lewat <code>hitungNilai()</code> , jadi datanya tidak mungkin tidak konsisten.
Execution	Ketika <code>m.hitungNilai()</code> dipanggil, objek mana yang menjadi target operasi?	Objek yang ada di sebelah kiri titik atau panah, yaitu objek yang alamatnya dikirim diam diam sebagai <code>this</code> . Waktu <code>daftar[12]->hitungNilai()</code> dipanggil di eksperimen, cuma objek indeks 12 yang berubah, dua objek lain tidak tersentuh. Jadi satu kode method bisa bekerja pada 30 objek berbeda, yang membedakan cuma <code>this</code> .
Abstraction	Detail perhitungan mana yang disembunyikan di dalam method?	Rumus $0.30 * \text{tugas} + 0.30 * \text{uts} + 0.40 * \text{uas}$ disembunyikan di <code>hitungNilai()</code> , dan pemanggilnya bahkan tidak perlu tahu kapan method ini dipanggil karena constructor sudah mengurusnya. <code>tampilkan()</code> menyembunyikan format <code>setw</code> dan <code>setprecision</code> . <code>sortNilai()</code> dan ketiga filter menyembunyikan loop dan perbandingannya dari <code>main()</code> , sehingga <code>main()</code> cuma berisi urutan langkah tingkat tinggi.

4.4 Pertanyaan Pengayaan

1. Apa yang terjadi terhadap state objek Mahasiswa setelah method `hitungNilai()` dipanggil?

Atribut `nilaiAkhir` objek itu terisi atau tertimpa dengan hasil hitungan terbaru, atribut lain tidak berubah. Kalau `tugas`, `uts`, dan `uas` belum berubah sejak panggilan sebelumnya, hasilnya sama persis sehingga seolah tidak terjadi apa apa. Kalau salah satunya sudah berubah, barulah kelihatan. Di eksperimen, setelah `setUAS(50)` diikuti `hitungNilai()`, state objek indeks 12 berubah dari 94.50 jadi 76.10. Objeknya tetap objek yang sama, alamatnya juga sama, cuma isinya yang diperbarui.

2. Mengapa atribut `nilaiAkhir` lebih tepat ditempatkan sebagai bagian dari objek Mahasiswa daripada sebagai variabel global?

Karena nilai akhir itu milik satu mahasiswa tertentu, bukan milik program. Kalau jadi variabel global, 30 mahasiswa berebut satu kotak yang sama dan cuma nilai terakhir yang tersimpan, padahal `sortNilai()` butuh membandingkan 30 nilai sekaligus. Selain itu variabel global bisa diubah dari mana saja sehingga susah dilacak kalau salah. Sebagai atribut, tiap objek punya salinannya sendiri di alamat masing masing, terbukti dari tiga alamat berbeda di eksperimen bagian [1].

3. Apa konsekuensi jika seluruh atribut `private` atau `public` terhadap encapsulation?

Kalau semuanya `private`, data benar benar terlindungi tapi dari luar tidak ada yang bisa dibaca sama sekali, jadi tetap butuh getter supaya kelas lain bisa memakainya. Buktinya `sortNilai()` dan ketiga filter di `SistemAkademik` semua lewat `getNIM()`, `getNama()`, dan `getNilai()`. Kalau semuanya `public`, encapsulationnya hilang: kelas tidak bisa lagi menjamin `nilaiAkhir` sesuai rumus, dan kalau suatu saat rumusnya diganti, semua kode di luar yang menyentuh atribut langsung harus ikut diperiksa. Yang wajar ya campuran seperti kode dosen, data `private` dan operasi `public`.

4. Jika terdapat 30 objek Mahasiswa, apakah masing-masing objek memiliki state sendiri? Buktikan melalui eksperimen.

Punya. Eksperimennya ada di `kode/eksperimen/p2-state.cpp`, hasilnya di `hasil/eksperimen.txt` bagian [1] dan [2]. Tiga puluh objek dibuat, lalu alamat dan nilainya dicetak: objek 0 di `0x104a118f0` nilai 82.50, objek 12 di `0x104a130c0` nilai 94.50, objek 29 di `0x104a13500` nilai 90.00. Alamat berbeda berarti memorinya terpisah. Setelah itu cuma objek 12 yang UAS-nya diubah jadi 50 lalu dihitung ulang. Hasilnya objek 0 tetap 82.50, objek 29 tetap 90.00, dan objek 12 berubah jadi 76.10. Kalau statenya dipakai bersama, ketiganya pasti ikut

berubah.

5. Apa yang terjadi terhadap atribut sebuah objek ketika objek tersebut keluar dari scope?

Untuk objek yang dibuat biasa tanpa `new`, begitu keluar blok destructornya dipanggil dan memorinya dilepas, jadi atributnya ikut hilang. Terlihat di `hasil/eksperimen.txt` bagian [3], baris destructor objek 99001 muncul persis setelah blok ditutup dan sebelum baris sudah keluar blok. Untuk objek hasil `new`, yang keluar scope cuma pointernya, objeknya sendiri masih di heap dan atributnya masih ada. Kalau pointernya satu satunya jalan menuju objek itu, objeknya jadi tidak bisa dijangkau tapi memorinya tetap terpakai, itu memory leak.

```
**6. Bandingkan lifetime Mahasiswa m(...); dan Mahasiswa* m = new  
Mahasiswa(...);**
```

`Mahasiswa m(...)` disimpan di stack, umurnya mengikuti blok tempatnya dideklarasikan, dihancurkan otomatis dan urutannya kebalikan dari urutan pembuatan. `Mahasiswa* m = new Mahasiswa(...)` disimpan di heap, umurnya ditentukan programmer dan baru berakhir kalau `delete` dipanggil. Di eksperimen bagian [4] objek 99002 dibuat dengan `new` tanpa `delete` dan destructornya memang tidak pernah muncul di keluaran, sedangkan bagian [5] objek 99003 dihapus dengan `delete` dan baris destructor objek 99003 langsung muncul. Program dosen memakai `new` karena `SistemAkademik` menyimpan pointer, jadi objeknya harus tetap hidup setelah baris pembuatannya lewat. Konsekuensinya harus ada `delete` atau destructor di `SistemAkademik`, dan itu belum ada.

PROGRAM 3: PARADIGMA FUNCTIONAL DENGAN HASKELL

5.1 Kode yang Dilengkapi

Tiga fungsi yang dikosongkan di Program 3 sudah dipanggil di main, jadi GHC menolak dengan pesan Variable not in scope. Ketiganya ditulis dengan filter supaya tetap murni dan mengembalikan list baru, sesuai cara main memakainya lewat mapM_ tampilkan. Perhatikan tidak ada penghitung ketemu seperti di Program 1 dan 2, karena jumlah hasilnya cukup dibaca dari panjang list.

Kode 5.1. Pencarian dan penyaringan versi Haskell (program3.hs)

```
searchNIM :: String -> [Mahasiswa] -> [Mahasiswa]
searchNIM kunci =
    filter (\m -> nim m == kunci)

-- =====
-- 6. TAMBAHKAN SEARCH BERDASARKAN NAMA
-- =====

searchNama :: String -> [Mahasiswa] -> [Mahasiswa]
searchNama kunci =
    filter (\m -> nama m == kunci)

-- =====
-- 7. TAMBAHKAN FILTER BERDASARKAN NILAI
-- =====

filterNilai :: Float -> [Mahasiswa] -> [Mahasiswa]
filterNilai batas =
    filter (\m -> nilaiAkhir m >= batas)
```

5.2 Keluaran Program

Keluaran Program 3, bagian daftar nilai dan bagian hasil pencarian:

```
2026-09-21-praktikum-2-paradigma — ghaisan@Ghaisans-MacBook-Pro — ..m-2-paradigma — -z...

> ./build/program3 | sed -n '1,12p;33,45p'

=== NILAI TERTINGGI -> TERENDAH ===
24013 | Maya | Nilai: 94.5 | Grade: A
24026 | Zahra | Nilai: 92.600006 | Grade: A
24024 | Xena | Nilai: 92.0 | Grade: A
24008 | Hadi | Nilai: 91.600006 | Grade: A
24003 | Citra | Nilai: 90.5 | Grade: A
24030 | Tiara | Nilai: 90.0 | Grade: A
24019 | Sinta | Nilai: 89.8 | Grade: A
24005 | Eka | Nilai: 87.700005 | Grade: A
24011 | Karin | Nilai: 87.0 | Grade: A
24021 | Ulya | Nilai: 86.600006 | Grade: A

=== SEARCH NIM: 24013 ===
24013 | Maya | Nilai: 94.5 | Grade: A

=== SEARCH NAMA: Maya ===
24013 | Maya | Nilai: 94.5 | Grade: A

=== NILAI >= 80 ===
24001 | Andi | Nilai: 82.5 | Grade: B
24003 | Citra | Nilai: 90.5 | Grade: A
24005 | Eka | Nilai: 87.700005 | Grade: A
24008 | Hadi | Nilai: 91.600006 | Grade: A
24011 | Karin | Nilai: 87.0 | Grade: A

~/Doc/Ku/programming-pragmatics/p/2026-09-21-praktikum-2-paradigma on main !2 ?4 at 09:12:57
>
```

Gambar 5.1: Keluaran Program 3 versi Haskell

5.3 Tabel Pengamatan

Tabel 5.1: Hasil pengamatan Program 3

Aspek	Pengamatan spesifik	Jawaban
Binding	Kapan parameter <code>m</code> memperoleh sebuah nilai mahasiswa?	Waktu fungsinya dipakai pada satu elemen. Di filter (<code>\m -> nilaiAkhir m >= batas</code>) daftar, <code>m</code> terikat ke elemen pertama, dievaluasi, lalu terikat lagi ke elemen kedua, begitu seterusnya. Karena Haskell malas, bindingnya baru benar benar terjadi saat hasilnya dibutuhkan, misalnya waktu <code>mapM_</code> tampilkan mencetaknya. Bindingnya sekali pakai per elemen, bukan satu variabel yang ditimpa 30 kali.
Binding	Apakah terdapat assignment ulang terhadap <code>m</code> ?	Tidak ada, dan memang tidak bisa. Di seluruh <code>program3.hs</code> tidak ada satu pun tanda <code>=</code> yang berperan sebagai penimpaan nilai. Tanda <code>=</code> di Haskell artinya definisi, bukan assignment. Sekali <code>m</code> terikat ke satu mahasiswa, ikatan itu tidak berubah sampai evaluasinya selesai.
Scope	Di mana <code>m</code> pada lambda <code>\m -> ...</code> dapat digunakan?	Hanya di badan lambda itu, yaitu bagian setelah tanda panah sampai akhir ekspresinya. Di filter <code>Nilai</code> yang saya tulis, <code>m</code> cuma dipakai di <code>nilaiAkhir m >= batas</code> . Di luar itu namanya tidak dikenal.
Scope	Apakah <code>m</code> dapat digunakan di luar lambda?	Tidak. Dicoba di kode/eksperimen/p3-scope.hs dan GHC menjawab <code>error: [GHC-88464] Variable not in scope: m :: Mahasiswa</code> . Jadi sama seperti <code>i</code> pada <code>for</code> di Program 1, hanya saja alasannya lebih rapi: lambda membuat scope baru dan tidak ada variabel yang bocor keluar.

Aspek	Pengamatan spesifik	Jawaban
Lifetime	Kapan binding <code>m</code> dibutuhkan selama evaluasi filter?	Cuma selama satu elemen sedang diperiksa. Begitu predikatnya menghasilkan <code>True</code> atau <code>False</code> , binding itu tidak dipakai lagi dan yang berikutnya dibuat untuk elemen selanjutnya. Jadi ada 30 binding <code>m</code> yang umurnya pendek pendek, bukan satu binding yang hidup panjang. Karena evaluasinya malas, binding untuk elemen yang hasilnya tidak pernah dipakai bahkan tidak perlu dibuat sama sekali.
State	Bandingkan list mahasiswa sebelum dan sesudah filter. Apakah data awal berubah?	Tidak berubah. Di <code>hasil/eksperimen.txt</code> bagian [1] list awalnya <code>["24001", "24013", "24016", "24006"]</code> dengan panjang 4. Setelah difilter, bagian [2] menghasilkan <code>["24001", "24013"]</code> . Lalu bagian [3] mencetak list awal lagi dan isinya masih <code>["24001", "24013", "24016", "24006"]</code> dengan panjang tetap 4. Hal yang sama berlaku untuk sort di bagian [6]: hasil sortnya <code>["24013", "24001", "24006", "24016"]</code> tapi list asalnya tetap urut seperti semula.
State	Apakah <code>nilaiAkhir m</code> mengubah objek mahasiswa?	Tidak. Fungsi itu cuma membaca tugas <code>m</code> , <code>uts m</code> , dan <code>uas m</code> lalu mengembalikan satu <code>Float</code> . Di <code>hasil/eksperimen.txt</code> bagian [4], <code>nilaiAkhir</code> maya dipanggil dua kali dan keduanya 94.5, lalu <code>uas</code> maya dicek dan masih 96.0. Bandingkan dengan Program 1 dan 2 yang menyimpan hasilnya ke dalam data.
Evaluation	Identifikasi fungsi yang dievaluasi dalam ekspresi <code>map/filter/fold</code> .	Di <code>semuaNilai = map nilaiAkhir</code> , yang dievaluasi <code>nilaiAkhir</code> untuk tiap elemen. Di <code>filterNilai batas = filter (\m -> nilaiAkhir m >= batas)</code> , yang dievaluasi lambdanya, dan di dalamnya <code>nilaiAkhir</code> serta operator <code>>=</code> . Di <code>totalNilai</code> , yang dievaluasi lambda dua parameter <code>\m total -> nilaiAkhir m + total</code> bersama <code>foldr</code> . Di <code>sortNilai</code> , <code>comparing nilaiAkhir</code> dievaluasi setiap kali dua elemen dibandingkan. Semuanya baru dievaluasi kalau hasilnya dipakai, misalnya <code>print (rataRata mahasiswa)</code> yang memaksa seluruh rantai perhitungan dijalankan dan menghasilkan 77.80667.
Transformation	Tentukan input dan output dari <code>map nilaiAkhir mahasiswa</code> .	Inputnya <code>[Mahasiswa]</code> berisi 30 elemen, outputnya <code>[Float]</code> berisi 30 elemen juga. Hasil lengkapnya ada di <code>hasil/program3.txt</code> bagian <code>=== SEMUA NILAI AKHIR ===</code> , dimulai <code>[82.5, 75.0, 90.5, 64.5, ...]</code> . Urutannya persis mengikuti urutan list aslinya, bukan urutan terbesar, karena <code>map</code> tidak menyusun ulang apa apa. Tipenya berubah dari record jadi angka, jumlah elemennya tetap.
Higher-order function	Identifikasi fungsi yang menerima fungsi sebagai argumen.	<code>map</code> menerima <code>nilaiAkhir</code> , <code>filter</code> menerima <code>lambda</code> , <code>foldr</code> menerima <code>lambda</code> dua parameter, <code>sortBy</code> menerima fungsi pembandingan, dan <code>mapM_</code> menerima <code>tampilkan</code> . Ada juga <code>comparing</code> yang menerima <code>nilaiAkhir</code> lalu mengembalikan fungsi pembandingan, dan <code>flip</code> yang menerima fungsi pembandingan itu lalu mengembalikan versi terbaliknyanya. Jadi <code>flip (comparing nilaiAkhir)</code> menerima sekaligus mengembalikan fungsi.
Immutability	Cari statement yang mengubah nilai mahasiswa. Apakah ada?	Tidak ada satu pun. Di seluruh berkas tidak ada operasi yang menulis ke data yang sudah ada. Semua fungsi menghasilkan nilai baru: <code>nilaiAkhir</code> menghasilkan <code>Float</code> baru, <code>filter</code> dan <code>sortNilai</code> menghasilkan list baru, <code>map</code> menghasilkan list baru. Bahkan tidak ada sintaks untuk menulis ulang <code>uas m</code> , yang ada cuma membuat record baru. Karena itu di Program 3 tidak dibutuhkan variabel <code>temp</code> seperti di bubble sort Program 1.
Composition	Amati hasil jika filter dikombinasikan dengan <code>map</code> .	Dicoba di <code>hasil/eksperimen.txt</code> bagian [5] dengan <code>map nilaiAkhir (filter (\m -> nilaiAkhir m >= 80) daftar)</code> dan hasilnya <code>[82.5, 94.5]</code> . Yang terjadi: filter memilih dulu, baru <code>map</code> mengubah tipenya, jadi outputnya list angka bukan list mahasiswa. Yang menarik urutannya ikut list asal, bukan terbesar, karena tidak ada sort di rantai itu. Kalau urutannya dibalik jadi filter setelah <code>map</code> , tipenya tidak cocok lagi karena <code>nilaiAkhir</code> butuh <code>Mahasiswa</code> sedangkan hasil <code>map</code> sudah <code>Float</code> . Jadi komposisi di Haskell dibatasi tipe, bukan dibatasi state seperti di Program 1.

5.4 Pertanyaan Pengayaan

1. Mengapa `nilaiAkhir m` tidak perlu mengubah data mahasiswa untuk menghasilkan nilai akhir?

Karena nilai akhir bisa dihitung ulang kapan saja dari `tugas`, `uts`, dan `uas` yang sudah ada. Menyimpannya ke dalam data cuma ada gunanya kalau kita mau menghemat perhitungan, dan itu justru bikin data bisa basi seperti di Program 1. Di Haskell nilai akhir diperlakukan sebagai fungsi dari data, bukan sebagai bagian data. Perhatikan data Mahasiswa di Program 3 memang tidak punya field `nilaiAkhir`, beda dengan `struct Mahasiswa` di Program 1.

2. Apa perbedaan antara `nilaiAkhir m` dan `m.nilaiAkhir = ...`?

`nilaiAkhir m` adalah pemanggilan fungsi yang menghasilkan nilai baru dan tidak menyentuh `m.m.nilaiAkhir = ...` adalah assignment yang menempa isi memori milik `m`. Yang pertama bisa dipanggil sepuluh kali tanpa efek samping apa pun, yang kedua meninggalkan jejak permanen dan urutannya terhadap statement lain jadi penting. Yang pertama menjawab pertanyaan, yang kedua mengubah dunia.

3. Jika fungsi `nilaiAkhir` dipanggil dua kali dengan mahasiswa yang sama, apakah hasilnya selalu sama? Mengapa?

Selalu sama. Diuji di `hasil/eksperimen.txt` bagian [4], `nilaiAkhir` maya dipanggil dua kali dan keduanya mencetak 94.5. Alasannya fungsi itu murni, hasilnya cuma bergantung pada argumen dan tidak ada yang bisa mengubah argumen itu di antara dua panggilan. Ini yang disebut referential transparency: `nilaiAkhir` maya boleh diganti langsung dengan 94.5 di mana pun tanpa mengubah arti program. Di Program 1 jaminan seperti ini tidak ada, karena `mahasiswa[12].nilaiAkhir` bisa saja sudah diubah fungsi lain di antara dua pembacaan.

4. Bagaimana immutability dapat mengurangi risiko perubahan data yang tidak terduga?

Karena tidak ada operasi yang bisa mengubah data, sumber perubahan yang tidak terduga hilang dengan sendirinya. Kalau sebuah list isinya `["24001", "24013", "24016", "24006"]`, dia akan tetap begitu sampai kapan pun, tidak peduli fungsi apa yang dilewatinya. Waktu debugging kita cukup melihat definisi fungsinya, tidak perlu melacak urutan pemanggilan seperti yang harus dilakukan di Program 1. Kerugiannya boros memori karena tiap operasi bikin struktur baru, walaupun sebenarnya Haskell banyak berbagi bagian yang tidak berubah.

5. Apakah tidak adanya assignment ulang berarti program tidak melakukan komputasi? Jelaskan!

Bukan berarti begitu. Program 3 tetap menghitung 30 nilai akhir, mengurutkannya, memfilter 15 mahasiswa dengan nilai di atas 80, dan menghitung rata rata 77.80667, semuanya terlihat di

hasil/program3.txt. Bedanya komputasi dilakukan dengan menghasilkan nilai baru dari nilai lama, bukan dengan menimpa kotak yang sudah ada. Jadi assignment ulang itu salah satu cara berkomputasi, bukan satu satunya. Di paradigma imperative program maju dengan mengubah state, di paradigma functional program maju dengan menyusun dan mengevaluasi ekspresi.

5.5 Catatan Presisi Float

Waktu dijalankan, beberapa nilai tercetak sebagai 92.600006, 87.700005, 91.600006, 86.600006, dan 66.100006. Ini bukan salah rumus, tapi karena tipenya `Float` yang cuma 32 bit sehingga angka seperti 0.30 tidak bisa disimpan persis dalam biner, lalu `show` mencetak apa adanya. Program 1 dan 2 sebenarnya menyimpan galat yang mirip, cuma tidak kelihatan karena `setprecision(2)` membulatkan tampilannya jadi 92.60 dan 87.70. Kalau mau lebih tepat, tipenya bisa diganti `Double` atau tampilannya diformat dulu. Hal ini tidak mengubah urutan sort maupun grade, jadi kodenya saya biarkan seperti aslinya.

PROGRAM 4: LOGIC PROGRAMMING DENGAN SWI-PROLOG

6.1 Query Nomor 13 dan 14

Nomor 13, mahasiswa yang lulus tetapi tidak mendapat A:

Cara kerjanya `lulus(NIM)` dijalankan dulu supaya `NIM` terisi, baru `\+ grade(NIM, a)` mengecek bahwa goal itu gagal. Urutannya tidak boleh dibalik, karena `\+ grade(NIM, a)` dengan `NIM` masih kosong akan langsung gagal begitu ditemukan satu mahasiswa bergrade A, dan hasilnya jadi salah. Hasilnya 18 `NIM`, terekam di `hasil/program4.txt`: 24001, 24002, 24004, 24007, 24009, 24010, 24012, 24014, 24015, 24017, 24018, 24020, 24022, 24023, 24025, 24027, 24028, dan 24029. Angkanya cocok, dari 30 mahasiswa ada 2 yang tidak lulus yaitu Fajar 59.2 dan Putra 57.4, lalu 28 dikurangi 10 yang dapat A sama dengan 18.

Nomor 14, mahasiswa yang perlu bimbingan dan nilainya di bawah 60:

`nilai_akhir` dipanggil lagi supaya `Nilai` ikut ter-binding dan bisa dibandingkan dengan 60, karena `perlu_bimbingan/1` cuma mengembalikan `NIM` tanpa nilainya. Hasilnya 2 mahasiswa, terekam di `hasil/program4.txt`: 24006 Fajar dengan 59.20 dan 24016 Putra dengan 57.40. Masuk akal, `perlu_bimbingan` batasnya di bawah 65 dan menghasilkan 5 orang, setelah disaring lagi di bawah 60 tersisa 2 orang. Ini contoh query yang lebih ketat daripada rulenya sendiri, dan kita tidak perlu membuat rule baru untuk itu.

Kode 6.1. Dua query gabungan yang diminta dokumen

```
% 13. mahasiswa yang lulus tetapi tidak mendapat A
?- lulus(NIM), \+ grade(NIM, a).

% 14. mahasiswa yang perlu bimbingan dan nilainya di bawah 60
?- perlu_bimbingan(NIM), nilai_akhir(NIM, Nilai), Nilai < 60.
```

6.2 Keluaran Program

Keluaran kedua query tersebut waktu seluruh daftar query dijalankan sekaligus:

```

> swipl -q -g demo -t halt kode/program4-prolog/program4.pl | sed -n '100,124p'
NIM = 24030

--- 13. lulus(NIM), \+ grade(NIM, a) ---
NIM = 24001
NIM = 24002
NIM = 24004
NIM = 24007
NIM = 24009
NIM = 24010
NIM = 24012
NIM = 24014
NIM = 24015
NIM = 24017
NIM = 24018
NIM = 24020
NIM = 24022
NIM = 24023
NIM = 24025
NIM = 24027
NIM = 24028
NIM = 24029

--- 14. perlu_bimbingan(NIM), nilai_akhir(NIM, Nilai), Nilai < 60 ---
NIM = 24006, Nilai = 59.20
NIM = 24016, Nilai = 57.40
  
```

Gambar 6.1: Hasil query nomor 13 dan 14

6.3 Tabel Pengamatan

Tabel 6.1: Hasil pengamatan Program 4

Aspek	Pengamatan spesifik	Jawaban
Binding	Jalankan mahasiswa("24013", Nama, Tugas, UTS, UAS). Catat binding setiap variabel.	Hasilnya di hasil/program4-pengamatan.txt: Nama = Maya, Tugas = 95, UTS = 92, UAS = 96. Keempatnya memperoleh nilai sekaligus saat query berhasil diunifikasikan dengan fakta mahasiswa("24013", "Maya", 95, 92, 96). Argumen pertama sudah terisi sejak awal sehingga berfungsi sebagai penyaring, empat sisanya kosong dan justru diisi oleh faktanya. Cuma ada satu solusi karena cuma ada satu fakta dengan NIM itu.
Binding	Jalankan grade(NIM, a). Identifikasi kapan NIM memperoleh nilai.	NIM masih kosong waktu query dikirim. Dia baru terisi jauh di dalam, waktu nilai_akhir(NIM, Nilai) memanggil mahasiswa(NIM, _, Tugas, UTS, UAS) dan berhasil diunifikasikan dengan sebuah fakta. Jadi bindingnya berjalan dari dalam ke luar: fakta mengisi NIM dan ketiga komponen nilai, Nilai dihitung dengan is, baru dibandingkan dengan 85. Kalau perbandingannya gagal, binding itu dibatalkan dan dicoba fakta berikutnya.
Scope	Bandingkan variabel NIM pada nilai_akhir/2 dan grade/2. Apakah keduanya merupakan variabel yang sama?	Bukan. Keduanya kebetulan dieja sama, tapi scope variabel di Prolog sebatas satu klausa. NIM di badan grade/2 adalah variabel milik klausa grade, sedangkan NIM di kepala nilai_akhir/2 milik klausa nilai_akhir. Keduanya terhubung bukan karena namanya sama, tapi karena diunifikasikan saat pemanggilan. Kalau NIM di grade diganti jadi x semuanya, programnya tetap jalan sama persis.

Aspek	Pengamatan spesifik	Jawaban
Scope	Amati scope <code>NIM</code> di dalam satu rule.	Di dalam satu klausa, satu nama variabel berarti satu variabel yang sama, mulai dari kepala sampai titik penutup. Di <code>aman(NIM) :- lulus(NIM), nilai_akhir(NIM, Nilai), Nilai >= 75</code> . ketiga <code>NIM</code> itu satu variabel, jadi begitu <code>lulus(NIM)</code> mengisinya, <code>nilai_akhir</code> langsung memakai nilai yang sama. Inilah yang bikin rule bisa menyaring, kalau namanya beda semua hasilnya jadi salah.
Lifetime	Jalankan <code>grade(NIM, a)</code> lalu tekan ;. Amati perubahan solusi/binding.	Binding cuma hidup selama satu solusi. Begitu ditekan ;, Prolog membatalkan binding terakhir lalu mencari fakta berikutnya yang cocok. Urutan solusinya terekam di <code>hasil/program4-pengamatan.txt</code> : solusi ke-1 sampai ke-10 berturut turut 24003, 24005, 24008, 24011, 24013, 24019, 24021, 24024, 24026, dan 24030, total 10 solusi. Jadi tidak ada satu variabel <code>NIM</code> yang ditimpa sepuluh kali, yang ada sepuluh binding pendek yang dipasang lalu dilepas.
Backtracking	Catat binding solusi pertama dan solusi berikutnya.	Solusi pertama <code>NIM = "24003"</code> yaitu Citra dengan nilai 90.5, solusi kedua <code>NIM = "24005"</code> yaitu Eka dengan nilai 87.7. Urutannya mengikuti urutan fakta di berkas, bukan urutan nilai dari besar ke kecil, karena Prolog mencoba fakta dari atas ke bawah. Ini beda dengan <code>hasil/program1.txt</code> yang hasilnya sudah diurutkan berdasarkan nilai.
State	Apakah fakta <code>mahasiswa(...)</code> berubah setelah query?	Tidak. Diuji dengan <code>aggregate_all(count, mahasiswa(_,_,_,_), Jumlah)</code> sebelum dan sesudah query, hasilnya sama sama 30 seperti terlihat di <code>hasil/program4-pengamatan.txt</code> . Query di Prolog cuma bertanya, tidak menulis. Fakta baru berubah kalau kita memakai <code>assert</code> atau <code>retract</code> , dan keduanya tidak dipakai di program ini.
Execution	Apakah Prolog menjalankan statement dari atas ke bawah seperti imperative?	Tidak sama. Prolog memang mencoba klausa dari atas ke bawah dan goal dalam satu badan dari kiri ke kanan, jadi sekilas mirip. Bedanya kalau sebuah goal gagal, Prolog tidak berhenti melainkan mundur ke titik pilihan terakhir dan mencoba kemungkinan lain, sesuatu yang tidak ada di C++. Selain itu tidak ada urutan statement yang mengubah state, jadi menukar <code>lulus(NIM)</code> dan <code>nilai_akhir(NIM, Nilai)</code> di <code>aman/1</code> tidak mengubah hasil, cuma jalur pencariannya. Bandingkan dengan Program 1 yang hasilnya berubah total kalau <code>hitungNilai()</code> dan <code>sortNilai()</code> ditukar.
Inference	Telusuri bagaimana <code>grade(NIM, a)</code> dapat diperoleh dari mahasiswa melalui <code>nilai_akhir</code> .	Penelusuran untuk 24013 saya cetak di <code>hasil/program4-pengamatan.txt</code> : fakta <code>mahasiswa("24013", _, 95, 92, 96)</code> ditemukan, rule 1 menghitung <code>nilai_akhir("24013", 94.50)</code> , lalu rule 3 mengecek <code>94.50 >= 85</code> dan terpenuhi sehingga <code>grade("24013", a)</code> benar. Yang penting di sini tidak ada satu pun fakta yang menuliskan bahwa Maya dapat A, kesimpulan itu diturunkan dari fakta mentah lewat dua rule.
Unification	Identifikasi pasangan term/variabel yang berhasil disatukan.	Pada query <code>mahasiswa(NIM, "Maya", Tugas, UTS, UAS)</code> hasilnya <code>NIM = "24013", Tugas = 95, UTS = 92, UAS = 96</code> . Yang disatukan: variabel kosong <code>NIM</code> dengan konstanta <code>"24013"</code> , konstanta <code>"Maya"</code> dengan konstanta <code>"Maya"</code> yang kebetulan sama sehingga cocok, lalu tiga variabel sisanya dengan tiga angka. Kalau argumen kedua diisi <code>"Mayaa"</code> , unifikasinya gagal dan semua binding dibatalkan. Simbol <code>_</code> di <code>nilai_akhir</code> juga bentuk unifikasi, cocok dengan apa saja tapi nilainya tidak disimpan.
Search	Amati bagaimana Prolog mencari solusi lain setelah ;.	Prolog kembali ke titik pilihan terakhir, yaitu fakta mahasiswa mana yang tadi dipakai, lalu melanjutkan dari fakta sesudahnya. Buktinya urutan solusi <code>perlu_bimbingan(NIM)</code> di <code>hasil/program4-pengamatan.txt</code> adalah 24004, 24006, 24009, 24016, 24027, persis urutan faktanya di berkas dan bukan urutan nilai. Jadi pencariannya depth first dengan backtracking. Kalau sudah tidak ada fakta tersisa, query berhenti dan menjawab false.

Aspek	Pengamatan spesifik	Jawaban
Abstraction	Tentukan rule mana yang merepresentasikan pengetahuan domain.	Hampir semua rule di BAGIAN 2. <code>nilai_akhir/2</code> memuat aturan bobot 30/30/40, <code>lulus/1</code> memuat batas kelulusan 60, <code>grade/2</code> memuat lima batas <code>grade</code> , dan <code>perlu_bimbingan/1</code> , <code>unggul/1</code> , <code>aman/1</code> , serta <code>nilai_tinggi/1</code> memuat kebijakan akademik lain. Rule seperti <code>sangat_baik/1</code> dan <code>mahasiswa_prioritas/1</code> tidak menambah pengetahuan baru, cuma memberi nama lain untuk rule yang sudah ada. Bagusnya di Prolog pengetahuan ini ditulis sekali sebagai aturan, dan pertanyaan apa pun tinggal disusun dari aturan itu tanpa perlu menulis langkah pencariannya.

6.4 Pertanyaan Pengayaan

1. Apa perbedaan antara `mahasiswa("24013", "Maya", 95, 92, 96)` **dan** `nilai_akhir(NIM, Nilai) :- ... dalam hal pengetahuan yang direpresentasikan?`

Yang pertama fakta, yaitu pengetahuan yang langsung benar tanpa syarat dan harus ditulis satu per satu untuk 30 mahasiswa. Yang kedua rule, yaitu pengetahuan bersyarat yang berlaku untuk semua NIM sekaligus. Fakta menyimpan data mentah, rule menyimpan cara menurunkan informasi baru dari data itu. Karena rule ditulis sekali, kalau bobotnya berubah dari 30/30/40 jadi 20/30/50 cukup satu baris yang disunting dan semua kesimpulan ikut berubah, termasuk `grade`, `lulus`, dan `berprestasi`.

2. Pada query `?- mahasiswa(NIM, "Maya", Tugas, UTS, UAS)` **variabel mana saja yang memperoleh binding?**

Empat variabel: `NIM`, `Tugas`, `UTS`, dan `UAS`. Hasilnya terekam di `hasil/program4-pengamatan.txt` sebagai `NIM = "24013"`, `Tugas = 95`, `UTS = 92`, `UAS = 96`. Argumen kedua `"Maya"` bukan variabel melainkan konstanta, jadi dia tidak memperoleh binding tapi bertugas menyaring fakta mana yang boleh cocok. Ini menunjukkan predikat yang sama bisa dipakai dua arah: `NIM` dicari dari nama, atau nama dicari dari `NIM`, tanpa menulis dua fungsi berbeda seperti `searchNIM` dan `searchNama` di Program 1 dan 3.

3. Mengapa `NIM` **pada dua rule berikut tidak berarti variabel yang sama?**
`nilai_akhir(NIM, Nilai) :- ... dan grade(NIM, a) :- ...`

Karena scope variabel di Prolog sebatas satu klausa, dari kepala sampai titik penutupnya. Nama `NIM` di dua klausa berbeda adalah dua variabel berbeda yang kebetulan dieja sama, mirip parameter `x` di dua fungsi berbeda di C++. Mereka baru terhubung saat `grade` memanggil `nilai_akhir` dan argumennya diunifikasikan. Buktinya kalau seluruh `NIM` di klausa `grade` diganti jadi `x`, program tetap memberi sepuluh solusi yang sama.

4. Apa yang terjadi terhadap binding ketika mahasiswa menekan ; setelah memperoleh satu solusi?

Binding solusi sebelumnya dibatalkan seluruhnya, lalu Prolog mundur ke titik pilihan terakhir dan mencoba kemungkinan berikutnya. Jadi bukan binding lama ditimpa nilai baru, tapi benar benar dilepas dulu baru dipasang yang baru. Terlihat dari urutan solusi `grade(NIM, a)` di `hasil/program4-pengamatan.txt` yang berpindah dari 24003 ke 24005 lalu ke 24008 dan seterusnya sampai 24030. Kalau sudah tidak ada kemungkinan tersisa, Prolog menjawab false.

5. Mengapa Prolog dapat menghasilkan beberapa solusi dari `?- grade(NIM, a).`?

Karena `NIM` dikirim dalam keadaan kosong, jadi Prolog boleh mencoba semua 30 fakta mahasiswa. Setiap fakta yang lolos syarat `Nilai >= 85` menjadi satu solusi. Ada 10 fakta yang lolos, yaitu 24003, 24005, 24008, 24011, 24013, 24019, 24021, 24024, 24026, dan 24030. Angka 10 ini cocok dengan jumlah mahasiswa bergrade A di `hasil/program3.txt`. Dengan kata lain query ini sekaligus jadi pencarian, tanpa kita perlu menulis loop seperti di `searchNilai()` pada Program 1.

LESSON LEARNED PROGRAM 1 SAMPAI PROGRAM 4

Yang paling kelihatan, keempat program menjawab persoalan yang sama persis tapi cara berpikirnya beda beda. Nilai tertinggi Maya 94.50, terendah Putra 57.40, rata rata 77.80667, dan 10 mahasiswa bergrade A, angka ini sama di keempatnya. Yang berbeda bukan hasilnya, tapi apa yang harus saya tulis untuk sampai ke situ. Di Program 1 saya menulis langkahnya, di Program 2 saya menulis siapa yang bertanggung jawab atas apa, di Program 3 saya menulis transformasinya, di Program 4 saya menulis aturannya lalu tinggal bertanya.

Program 1 mengajarkan bahwa urutan statement itu bagian dari kebenaran program. Waktu saya coba menukar `hitungNilai()` dan `sortNilai()` hasilnya langsung kacau, indeks 0 malah diisi Putra dengan nilai 57.40. Kodenya tidak error, compilernya diam saja, programnya jalan normal, tapi jawabannya salah. Dari sini saya paham kenapa scope dan lifetime penting dipelajari: variabel yang hidup terlalu lama seperti array mahasiswa bisa diubah siapa saja, sedangkan `temp` yang hidup cuma satu blok jauh lebih aman. Saya juga sempat kaget waktu `nilaiAkhir` ternyata 0.00 sebelum `hitungNilai()` dipanggil, ternyata itu karena aggregate initialization cuma diisi lima anggota.

Program 2 mengajarkan bahwa encapsulation itu soal menjaga aturan, bukan soal menyembunyikan. Waktu saya coba akses `m.nim` dari `main()` dan g++ menolak, awalnya terasa merepotkan. Tapi setelah lihat bahwa `nilaiAkhir` cuma bisa diisi lewat `hitungNilai()`, saya paham maksudnya: kelas menjamin datanya selalu konsisten dengan rumus. Bandingkan dengan Program 1 yang siapa saja bisa menulis `mahasiswa[0].nilaiAkhir = 100;` dan tidak ada yang mencegah. Pelajaran lain soal lifetime: destructor yang saya tambahkan menunjukkan objek `new` tanpa `delete` benar benar tidak pernah dihancurkan, dan kode dosen punya 30 objek seperti itu. Punya kendali penuh atas umur objek artinya juga punya kewajiban membereskannya.

Program 3 mengajarkan bahwa banyak masalah di Program 1 hilang sendiri kalau datanya tidak bisa diubah. Tidak ada `temp` karena tidak ada penukaran, tidak ada data basi karena nilai akhir dihitung saat dibutuhkan, tidak ada urutan pemanggilan yang harus dihafal karena tidak ada state yang ditinggalkan. Waktu saya cetak list awal sebelum dan sesudah filter dan sort, isinya tetap sama persis, padahal di Program 1 posisi Andi sudah pindah dari indeks 0 ke 10. Yang bikin saya agak kaget, di Haskell error justru datang lebih awal: waktu `searchNIM` belum saya tulis, GHC langsung menolak dengan `Variable not in scope` sebelum programnya sempat jalan. Saya juga belajar soal presisi `Float` dari angka 92.600006 yang muncul di keluaran, sesuatu yang di Program 1 tertutup `setprecision(2)` sehingga saya tidak pernah sadar galatnya juga ada di sana.

Program 4 mengajarkan bahwa kita bisa menulis pengetahuan tanpa menulis cara mencarinya. Ini yang paling beda dari tiga program sebelumnya. Di Program 1 saya harus membuat loop, variabel `ketemu`, dan pengecekan manual untuk `searchNilai()`. Di Prolog cukup menulis `nilai_tinggi(NIM) :- nilai_akhir(NIM, Nilai), Nilai >= 80.` lalu bertanya, dan mesin yang mengurus pencariannya lewat unifikasi dan backtracking. Waktu mengerjakan query nomor 13 saya baru paham kenapa urutan goal tetap penting: `\+ grade(NIM, a)` harus ditaruh setelah `lulus(NIM)` supaya NIM sudah terisi, kalau dibalik hasilnya salah. Jadi Prolog memang deklaratif, tapi tidak berarti urutan sama sekali tidak berpengaruh. Satu hal lagi, predikat `mahasiswa/5` bisa dipakai dua arah, cari nama dari NIM atau cari NIM dari nama, tanpa menulis dua fungsi terpisah seperti `searchNIM` dan `searchNama` yang harus saya tulis dua kali di Program 1 dan 3.

Kesimpulan buat saya sendiri: tidak ada paradigma yang menang di semua keadaan. Program 1 paling dekat dengan cara kerja mesin dan paling gampang diprediksi performanya, tapi paling gampang salah kalau programnya membesar. Program 2 merapikan Program 1 dengan membatasi siapa yang boleh menyentuh data, harganya kode jadi lebih panjang dan urusan memori jadi tanggung jawab kita. Program 3 paling aman dari bug yang berhubungan dengan state dan paling enak diuji karena fungsinya bisa dipanggil berkali kali dengan hasil yang sama, tapi butuh waktu untuk terbiasa berpikir tanpa variabel. Program 4 paling singkat untuk soal yang bentuknya pencarian dan penalaran, tapi tidak cocok untuk hal yang butuh langkah berurutan. Yang menarik, konsep binding, scope, lifetime, dan state ternyata ada di keempatnya, cuma wujudnya beda: di C++ berupa alamat memori yang bisa ditimpa, di Haskell berupa ikatan sekali pakai, di Prolog berupa binding yang dipasang dan dilepas mengikuti backtracking.